Evasion Is a Relation

A confidence-routed cascade for political question evasion, and what it measures

SemEval-2026 Task 6 (CLARITY)

Vidvathama R Siddarth Gottumukkula Sanjana Reddy Vonteri Shashikanta Sahoo

Team Nier_ANLP

International Institute of Information Technology, Hyderabad

Advanced NLP ▪ October 2026

The task: how clearly does a politician answer each sub-question?

Each item pairs one **sub-question** from a journalist's turn, the **full question** and the politician's **answer**. We predict the 9-way label and map it to the 3-way one.

Subtask 1	Subtask 2	train
Clear Reply	Explicit	30.5%
Ambivalent	Implicit, Dodging, General, Deflection, Partial/half-answer	59.2%
Clear Non-Reply	Declining to answer, Claims ignorance, Clarification	10.3%

Data and metric

- Train 3,448 rows. Dev 308 items (3 annotators). Test 237 (2 annotators, labels unreleased)
- Macro-F1 over 9 classes. A prediction counts if *any* annotator gave it
- Every score in this talk is on dev

Why it is hard

- 69% of rows share their answer with another sub-question
- Humans disagree: one annotator vs. the other two scores **0.684**

What are we classifying? Three real dev items

Clear Reply → Explicit

Sub-question: "Do you stand by your figure, 30,000?"

Answer: *"You know, I stand by the figure. A lot of innocent people have lost their life—600,000, or whatever they guessed at, is just—it's not credible. . . . "*

Why this label: he states the requested position outright.

Ambivalent → Dodging

Sub-question: "Do you see a risk of triggering a wider war?"

Answer: *"I thought you were going to ask me about the pig."*

Why this label: he ignores the question and changes the subject.

Clear Non-Reply → Declining

Sub-question: "Is it a fair assumption that the CIA operatives are in Pakistan, cooperating with the Pakistanis, and that they're sharing everything with you and vice versa?"

Answer: *"John, what's fair is, I'm not—what you must assume is that I'm not going to talk about ongoing intelligence matters. . . . "*

Why this label: he openly refuses to give the information.

- All three annotators agree on each of these. The "pig" answer is attached to all three sub-questions of its turn, so the model must work out *which part* of an answer responds to *which* sub-question.
- We predict the 9-way evasion type (Subtask 2). The 3-way clarity level (Subtask 1) follows from it.

Shared-task results: LLM pipelines won, fine-tuned encoders reached about 0.50

Rank	Team	Approach (S2 ranking)	S2 test
1	**TeleAI**	DeepSeek-V3, 3 confidence-routed CoT stages	**0.68**
2	ChulaNLP	fine-tuned DeBERTa-large top-5 labels → few-shot Kimi-K2	0.61
3	CLaC	prompted LLMs, ahead of their fine-tuned encoders	0.59
12	PFW	50 DeBERTa models averaged (5 folds × 10 seeds)	0.50
Organiser baseline		fine-tuned Llama-70B	0.57

- 33 teams entered S2, with a median of about 0.52. Encoders level off near **0.50**.
- TeleAI on dev: 0.617 S2 / 0.812 S1, with 1.94 LLM calls and ∼7.3k input tokens per item.

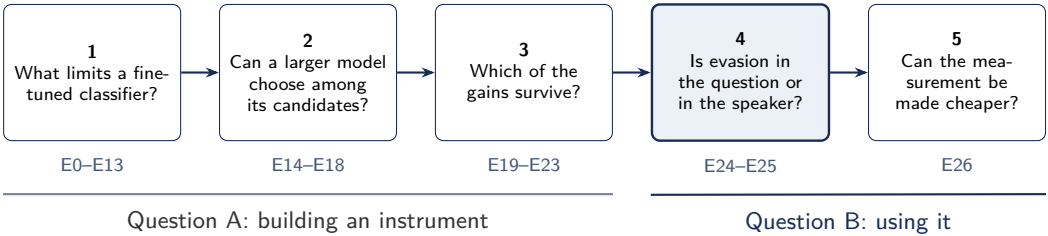
Test scores (237 items) are from Table 3 of the task overview paper. TeleAI's dev numbers are from its system paper. Our numbers are on dev.

Our two questions

A. Why do fine-tuned classifiers stop where they do?

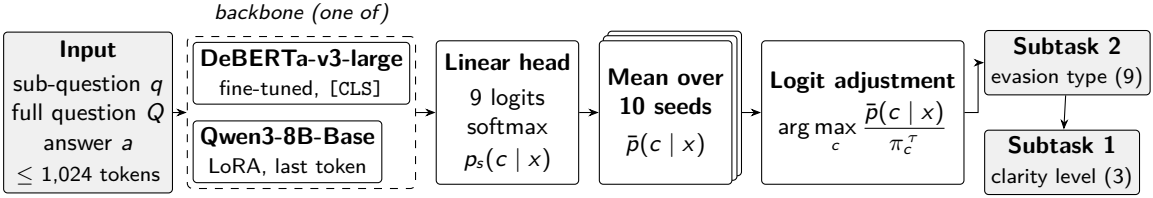
B. Once we have a good classifier of evasion, what can it tell us about evasion itself?

Overview: five stages, each asking one question



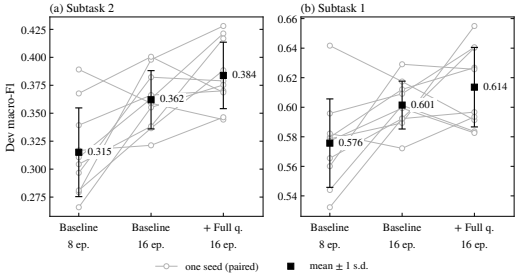
- Each experiment's plan, prediction and decision rule was written down *before* the run. Results we did not register are marked **exploratory**.
- Each result slide shows what we **predicted** next to what we **found**, including the many predictions that failed.
- Claims rest on 10 paired seeds. Parameters fitted on dev are scored by nested cross-validation, and the one choice made on dev after a failure ($\delta = 0.5$ for the distilled point) is flagged.
- From Stage 4 on, one frozen system labels everything, so no parameter changes between experiments.

The classification pipeline shared by both backbones



- **Backbone (one of two):** DeBERTa-v3-large, fully fine-tuned, reads the [CLS] state. Qwen3-8B-Base with LoRA reads the last token. Both see the sub-question q, the full question Q and the answer a ($\leq$ 1,024 tokens).
- **Head:** a new linear layer gives 9 logits and a softmax $p_s(c \mid x)$ over the evasion types for each seed s.
- **System:** average the 10 seeds into $\bar{p}(c \mid x)$, then divide by the train class prior π_c raised to τ.
- **Output:** the Subtask 2 label (9 types) fixes the Subtask 1 clarity level (3) through the taxonomy.

A controlled DeBERTa baseline and seed ensembling

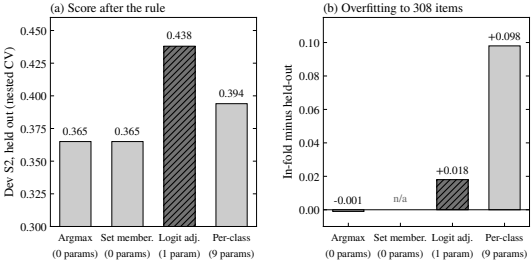


Seeds: the ten models differ only in their random seed. Averaging their softmax outputs reduces the variance between runs, and every claim in this talk compares paired seeds.

Result: 16 epochs plus the full question raised the model from **0.315 to 0.384** (+0.069, 9 of 10 seeds up). The 10-seed system with logit adjustment scores 0.405 S2 and 0.648 S1.

Takeaway: undertraining hid the gain (at 8 epochs, 0.282 against 0.337, with the loss still at 1.3–1.5), so we report fit next to every score.

The decision rule: logit adjustment



How it works

An argmax picks the class with the highest averaged probability. Macro-F1 weights all nine classes equally, but the train labels are dominated by the Ambivalent types (59%) and Explicit (31%). Dividing $\bar{p}(c \mid x)$ by π_c^{τ} lowers the score of frequent classes, so rare ones are predicted more often. With $\tau = 0$ this is the plain argmax, and $\tau = 1$ removes the train prior entirely. τ is the only fitted parameter, chosen on dev by nested cross-validation.

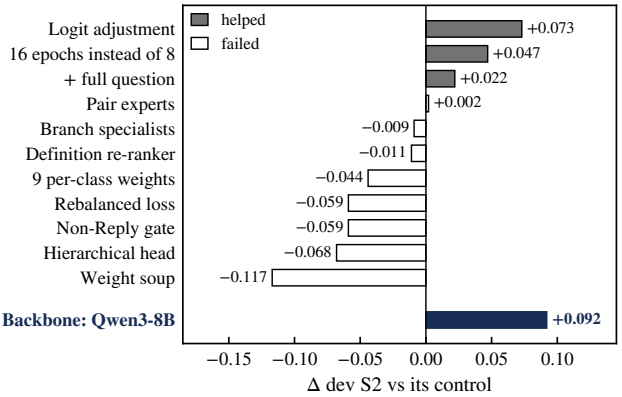
E4 Logit adjustment against per-class weights

Why: nine free class weights could fit the metric better than one scalar.
Result: logit adjustment 0.365 → **0.438**. Nine weights reach only 0.394, with an in-fold gain of +0.098 that does not transfer.
Takeaway: one scalar beats nine weights on 308 items. Every system uses it.

Structural changes failed, while the decision rule, training length and full question helped

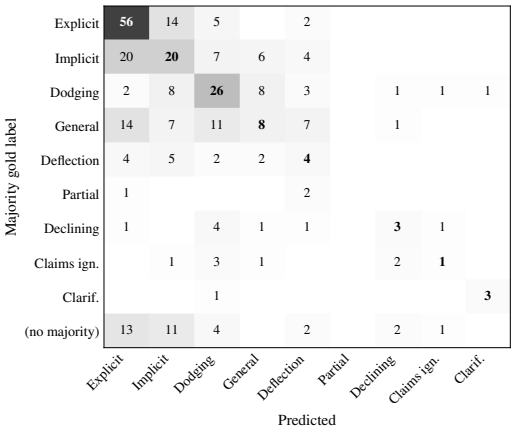
Four explanations for the plateau (DeBERTa-v3-large)

- **Noisy labels?** No: 53 errors were on items all annotators agreed on.
- **The decision rule?** A little: logit adjustment +0.073.
- **Label meaning?** No: a re-ranker given label definitions −0.011, though gold was in the top 3 for 90%.
- **Missing knowledge?** → tested by changing the backbone.



Takeaway: no structural change moved the system, so we changed the model itself.

Where the DeBERTa ensemble fails



How to read it

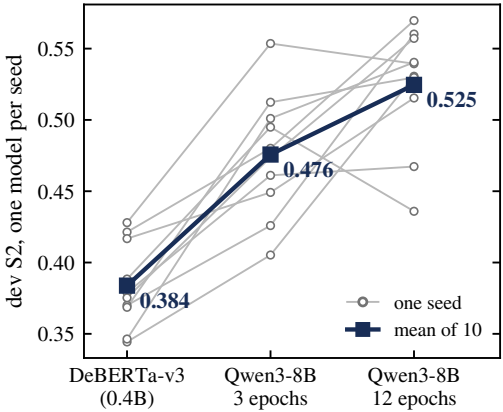
- Rows are the majority gold label (at least 2 of 3 annotators), plus a "no majority" row (33 items).
- Columns are the argmax of the 10-seed ensemble. The diagonal is right for 121 of 275 majority items.
- Scoring never uses the majority. A prediction counts if *any* annotator gave it.

What it tells us

- Errors sit inside *Ambivalent*. Implicit is predicted as Explicit 20 times, and General is spread over five other labels.
- Partial/half-answer is never predicted.
- The encoder fails on **how committed** an answer is, the same pairs annotators disagree on.

Takeaway: a model with more knowledge should improve these boundaries. We test this by changing only the backbone.

A stronger pretrained backbone gained more per model than all encoder changes combined



One change: DeBERTa-v3-large → Qwen3-8B-Base + LoRA, with the same rows, input, loss and selection, on 10 paired seeds.

Expected: gains on the Explicit–Implicit–General boundaries of the confusion matrix

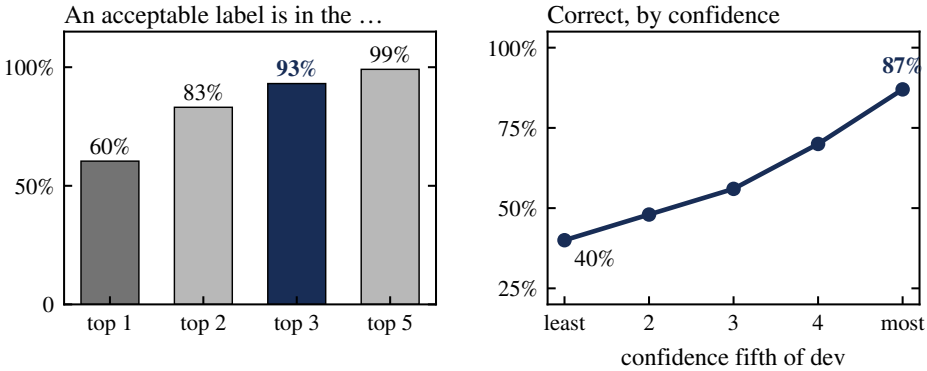
Found: +0.092 per model, 10 of 10 seeds up (all encoder changes together: +0.069)

Found: the gain is broad, largest on the Non-Reply types (Claims ignorance +0.40), so the expected location did not hold ✗ failed

"Stronger backbone", not "knowledge" alone: size, pretraining, LoRA and the head all changed together.

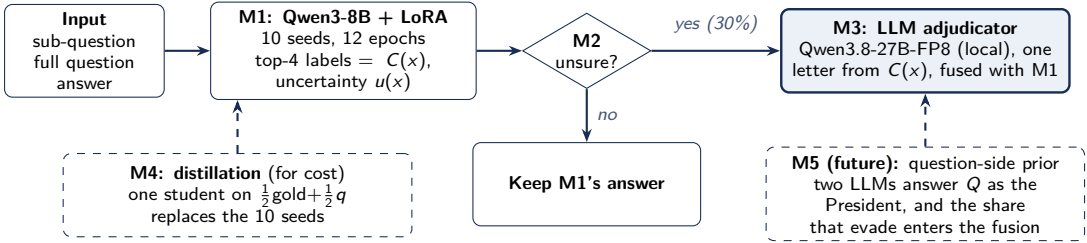
What it raised: the 8B model ranks the right label far better than it picks it.

The classifier ranks the right label better than it picks it



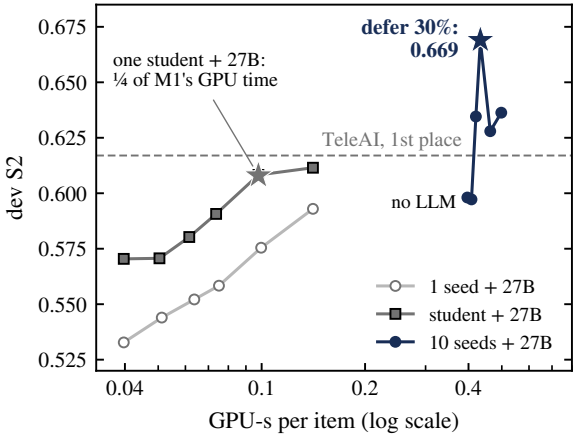
Takeaway: errors are mostly a choice among a few top candidates, made where the model is unsure. So let a larger model choose among them, *only* for the least confident items.

System design: a classifier, a router and an LLM adjudicator



- Registered checks that set the design: choosing at random inside M1's top 3 *lowers* S2 (0.587 vs 0.598), so the LLM is **fused** with M1, not substituted. Candidate sets by probability mass did not transfer to new rows, but sets by rank did (**top 4**).
- On 300 cross-fitted train rows (fused accuracy), retrieved examples *hurt* the 27B (0.330 vs 0.353 without), and a 304B model did worse than the local 27B (0.317): short prompt, local model, one output token.

Deferral rate against cost on dev



- **Defer 30%**: **0.669**, +0.067 over M1 [+0.009, +0.124], parameters held fixed, with 33 items fixed and 9 broken (after we fixed a bug in our own fusion code).
- **Defer everything**: worse (0.609 over ten splits). Where M1 is confident, the LLM is the weaker reader.
- **Distil**: one student plus the 27B matches the ten-seed M1 at a quarter of its GPU time. Ten students add only +0.005 (ten M1 seeds: +0.065).

What it raised: was 0.669 luck? One ensemble, parameters fitted on 308 items.

One answer, two sub-questions, two labels

One journalist's turn at a President's news conference, and one answer

"Is that a make-or-break issue for you, in terms of domestic politics? There's a Republican in Pennsylvania who says he doesn't think the troops should ... would you campaign for Mike Fitzpatrick?"

"I already have."

Explicit (3 of 3 annotators)

Sub-question: Would you campaign for Mike Fitzpatrick?

Answer: *I already have.*

Why this label: the answer states what was asked.

Dodging (3 of 3 annotators)

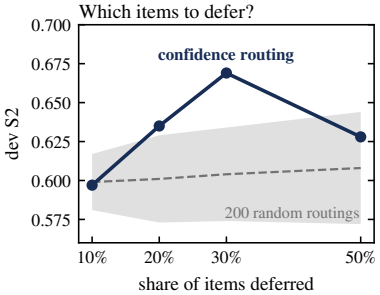
Sub-question: Is that a make-or-break issue for you, in terms of domestic politics?

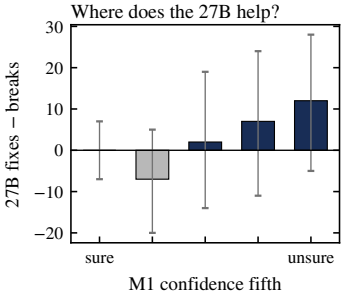
Answer: *I already have.*

Why this label: the answer says nothing about whether the issue is make-or-break.

Takeaway: 69% of training rows share their answer with another sub-question, so one answer can receive two labels. Whether an answer evades then depends on the question it is paired with. Stage 4 tests this, and the last slides return to this answer.

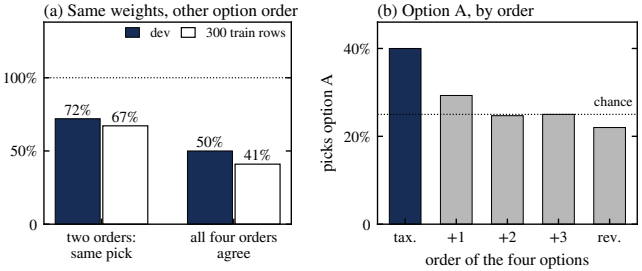
Controls on the cascade: routing, fusion and where the 27B helps





- On 3,448 cross-fitted train rows the cascade gains +0.014 [+0.002, +0.026], less than on dev, where its parameters were fitted.
- No random routing of the same size matches confidence routing (0.669 against 0.604 ± 0.015 over 200 draws). The 27B alone over all nine labels scores 0.591, below M1 (0.598), so fusion is needed.
- The 27B fixes about as many items as it breaks where M1 is confident and helps only where M1 is unsure. A learned fix/break router lost to confidence at 4 of 4 budgets.

The 27B prioritises option A



- LLMs choosing among lettered options are known to prefer some positions (Zheng et al., ICLR 2024). Ours prioritises **option A**: it picks A for 40% of items in taxonomy order, against 22–29% in the other orders (chance is 25%).
- In taxonomy order slot A usually holds Explicit, so a preference for A is also a preference for Explicit.
- Two orders of the same four candidates give the same pick on only 72% of dev items.
- We therefore average four cyclic orders. This costs four calls per deferred item and moved the system by $+0.011$ at split 0 but only $+0.002$ over ten splits.

Takeaway: the final system averages four option orders, and all later numbers use that setting.

C. Zheng, H. Zhou, F. Meng, J. Zhou, M. Huang. Large Language Models Are Not Robust Multiple Choice Selectors. ICLR 2024.

The frozen system compared with the winning system

System (dev)	S2	S1	LLM calls	GPU-s
DeBERTa-v3-large, 10 seeds	0.405	0.648	0	–
M1: Qwen3-8B, 10 seeds	0.598	0.743	0	0.40
Accuracy point, M1 + 27B	**0.650**	**0.788**	1.43	0.54
Distilled point, 4 orders	**0.608**	**0.785**	2.30	0.27
one option order	0.599	0.784	0.57	0.10
TeleAI, 1st place	0.617	0.812	1.94	n/a

Nested-CV split 0 (0.639 over ten splits for the accuracy point). All records re-scored once on one serving setup. LLM calls are 27B calls per item, four per deferred item with four orders. GPU-s is inference time per item, all on one GPU. TeleAI's row is its own dev result, with ∼7.3k LLM input tokens against our ∼1.6k. Its GPU time is not published.

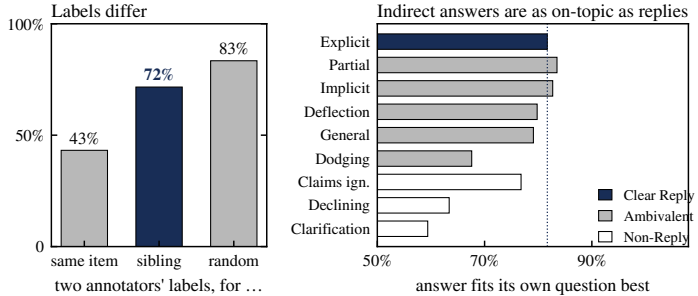
What it raised: if a label is not a property of the answer alone, what is it a property of?

Relation to the task's system papers

In the task's 30 system papers, these parts were already in use: candidate sets, LLM choosers, routing, distillation. **None tested whether such systems read the question.**

So we froze every setting and used the cascade as an **instrument**.

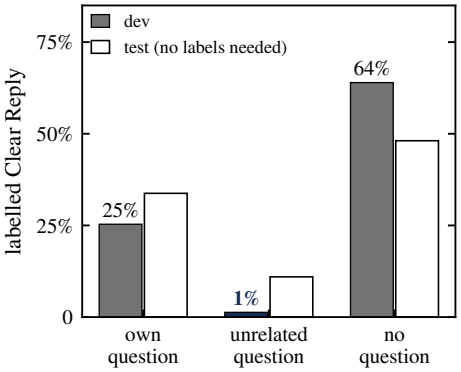
Labels of one answer across its sub-questions



- On dev, two sub-questions of one answer get different labels 72% of the time, against 83% for random items and 43% for two annotators on the same item. On train, 61.5% of 3,079 sibling pairs differ.
- Partial, Implicit, Deflection and General answers fit their question as well as Explicit ones, so relevance is not answerhood (Clear Reply vs rest: AUROC 0.566).

Takeaway: evasion is a relation between a question and an answer. The "I already have" answer is typical, not an exception.

Validity check: removing or swapping the question



A validity check before using it on new speakers: pair each answer with an unrelated question, or with none.

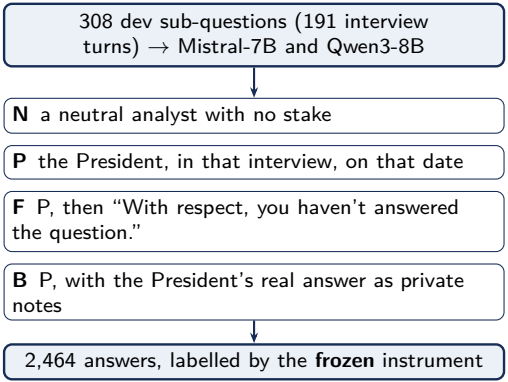
Predicted: the Clear-Reply share at least halves
Found: 25% → 1% ✓held

Surprise: with *no* question, Clear Replies *rise* to 64%. Without a question, nothing is evaded.

Our answer again, under an unrelated question

"Why did you keep saying there was a relationship between Saddam Hussein, Zarqawi, and Al Qaida . . . ?"
"I already have." → instrument: **Dodging**
With its own two questions it gives Explicit and Dodging, as all six annotations say.

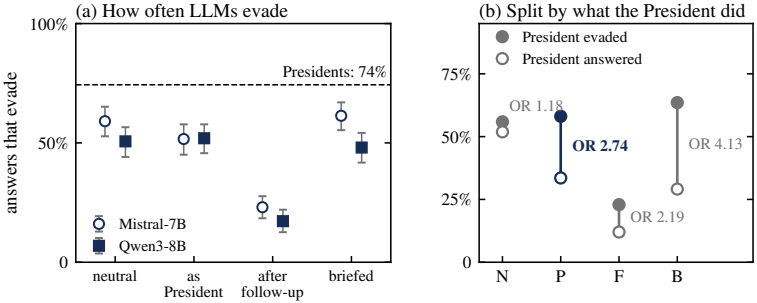
Does the question or the speaker drive evasion? Design



Registered predictions

- If the **question** drives evasion, a speaker with no stake (N) evades where the Presidents did: odds ratio ≥ 1.5.
- If the **stake** drives it, playing the President (P) raises the evasion rate by ≥ 0.10.
- **Pursuit** (F) turns $\geq 50\%$ of evasions into replies.
- **Notes** (B) help more on questions answered indirectly than on non-answers ($\geq +0.15$).

Evasion rates by role



No stake (N): odds ratio 1.18 [0.80, 1.73] ✗ failed

Stake raises the rate: −0.031 ✗ failed

President's role (P): odds ratio **2.74** [1.87, 4.17] exploratory

Pursuit (F): 66% of evasions become replies ✓ held

Notes (B): +0.113 [−0.006, +0.225], below the bar ✗ failed

In panel (b), the change is in the open dots. Where the President answered, a neutral analyst still evades 52%, but in the President's role the models evade only 34%.

What the role result shows and does not show

What it shows

- The role changes *where* models evade, not *how often* (52% vs 55%).
- In the President's role they answer directly what the Presidents answered directly. A neutral analyst, to whom the questions are not addressed, evades about half of them whatever the President did.
- So a question's answerability shows only for a speaker who can answer it.
- It holds for each model separately (Mistral 3.0, Qwen 2.5).

What it does not show

- It is exploratory, with two model families, one of them the judge's own (Qwen). Our data cannot separate answerability from strategic evasion.
- The P prompt adds identity, date and context, not only a stake, and the role odds ratio was not adjusted for answer length.
- The instrument was not human-validated on LLM text, though it is *more* confident there (seed agreement 0.81 vs 0.73).
- Anything about real politicians' intent.

What the evasion study tells us to act on

Everything so far comes from one pipeline: the classifier, the router, the 27B adjudicator, frozen as an instrument. Its results point to three things to act on.

1 Labels follow the question

One answer gets different labels under different sub-questions, and the instrument reads the question causally.

Act on it: always feed the full question with the answer. Never label an answer alone.

2 The question carries signal

LLMs in the President's role, who never saw the real answer, answer directly the questions the Presidents answered directly (odds ratio 2.74).

Act on it: no module uses this signal yet. A **question-side prior, M5**, could feed it to the fusion.

3 No shortcut

Half an answer agrees with the full-answer label on only 66% of items, and probes on the frozen model reach 0.397 S2 against 0.525 for one fine-tuned seed.

Act on it: keep the full model and make it cheaper instead. That is Stage 5.

Takeaway: the second point gives the next architectural element, M5, to which we return after Stage 5.

What it raised: can the full instrument run cheaper?

Cost of reduced precision: memory, throughput and latency

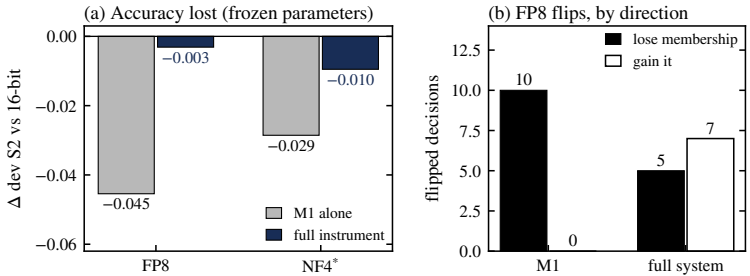
M1, 16-bit → reduced	FP8	NF4
Throughput (items/s)	43.1 → 35.9	7.7 → 6.3
ratio	**0.83×**	**0.82×**
Batch-1 latency, p50 (ms)	33.6 → 30.2	169 → 259
ratio	0.90×	**1.53×**
Peak memory (GB)	21.9 → 20.9	16.8 → 11.3
ratio	**0.96×**	**0.67×**
Label agreement (10 seeds)	96.1%	86.7%

FP8: native FP8 tensor cores of one RTX PRO 6000 (Blackwell), batch 32 for both. NF4: two RTX A4000, with 16-bit split over two cards at batch 8 and NF4 on one card at batch 16, so its speed is not like for like. Peak memory is per process. The weights of an 8B model alone take about 16, 8 and 4.5 GB in 16-bit, FP8 and NF4.

What we saw

- **FP8** ran on native FP8 tensor cores but was slower, not 1.3× faster as predicted, and barely smaller. Activations are quantized in unfused operations around each matrix product. Reported FP8 speed-ups rely on fused kernels, which we did not use.
- **NF4** saves memory (0.67×, short of the predicted 0.4×) but is slower. Its 4-bit weights are computed in bf16.
- Applied to the accuracy point's one-GPU cost, FP8's speed would raise it from 0.538 to 0.619 GPU-s per item (**+15%**).

Lower precision: accuracy lost and flipped decisions



- Frozen M1 in FP8 or NF4, each against 16-bit on the same GPU type. FP8: agreement 96.1% (bar 98%) ✗ failed speed 0.83× (bar 1.3×) ✗ failed flips fall where the 27B re-decides, 71% vs 36% by chance ✓ held
- All 10 of M1's membership-changing flips lose it, while the system's go both ways (5 lose, 7 gain), so with the same frozen parameters it loses 0.003 against M1's 0.045.
- *NF4: 27B records are missing for 60 of 136 deferred items and the system flips more decisions than M1 (57 vs 45): no conclusion.

Takeaway: precision is not a cost lever with these kernels.

Registered predictions that steered the work

1	Classifier	gain on the expected boundaries; gain larger on agreed items
2	Cascade	top-3 headroom; dev gain at 30%; 5 seeds $\approx$ 10; examples help; student at 30%; student ranks errors better
3	What survives	test text matters; learned router; LLM-knowledge distillation; cached prompt; escalation ladder
4	What it measures	reads the question; pursuit; probe less question-bound; question-driven (N); stake raises rate; notes; half answer suffices; frozen probe suffices
5	Precision	FP8 flips land where the 27B re-decides; FP8 fidelity; FP8 speed; NF4 memory (0.4$\times$)

held registered prediction held failed failed, and changed the next step

A selection of the registered predictions that steered the work. Every one, held or not, is in the experiment log.

Limitations, each with what we did and what comes next

Limitation	What we did	Next step
308 dev items, and test labels unreleased	10 paired seeds and nested CV	Score the frozen test predictions once labels are out, and test a second corpus
Not human-checked on LLM text	Label-free check: more confident on LLM text	Two annotators, 100 answers, the registered agreement bar
Two LLM families, and the role effect is exploratory	Per-model results	Add Llama and OLMo, register the role effect and adjust it for answer length
LLM decisions depend on option order and the serving stack	Four orders averaged, one serving setup	Repeat runs (none yet) and pin software and hardware
Quantization not faster as implemented	Same GPU type, and the same batch for FP8	Fused FP8 kernels, QLoRA for NF4, and every NF4-deferred item scored

Future work: a question-side evasion prior

Proposed module M5

- Every module reads the answer, but the role study suggests the question carries evasion signal of its own.
- For the items M2 defers, two open LLMs answer the question as the President, *without* the real answer.
- The frozen instrument labels those answers, and the share that evade enters the fusion as an extra term.
- Its weight is fitted on training rows, whose P-role answers must still be generated. The prediction will be registered before dev is scored.

Preliminary check on dev exploratory

P-role LLMs that evade	items	President evaded
0 of 2	101	61%
1 of 2	95	72%
2 of 2	112	**88%**

Added to M1 it helps only a little. AUROC for "the President evaded" goes from 0.872 to 0.875 on all items and from 0.722 to 0.746 on the 111 items M1 is least sure of.

Expected effect: a small gain on the items M1 is unsure about, mainly on the evade-or-not decision. It costs two extra generations per deferred item and needs the instrument to be valid on LLM text.

The check was written after the E25 results, on 308 dev items. It motivates a registered test and is not evidence for the module.

Conclusions

What we learned

1. **A stronger pretrained backbone, not more structure,** broke the classifier plateau, with +0.092 per model on 10 of 10 seeds.
2. **Rank, then adjudicate only where unsure.** The cascade reaches 0.65 dev S2, on par with the winner, with three quarters of its LLM calls. A distilled variant needs a quarter of the classifier's GPU time.
3. **Evasion is a relation.** Labels follow the question. In two LLMs, in the President's role, models answer directly what the Presidents answered directly (exploratory). This motivates M5, a question-side prior.

The shared answer, again

"I already have."

campaign for him? → Explicit
make-or-break? → Dodging
an unrelated question? → Dodging

The instrument gets all three, because it reads the pair.

Thank you. Questions?

Code, pre-registered log and every raw output

github.com/IamSid44/CLARITY-Unmasking-Political-Question-Evasions

Runs and adapters huggingface.co/siddarthg44/clarity-semeval26

Training curves wandb.ai/iamsid44-iiit-hyderabad/clarity-semeval26